SIMATS ENGINEERING

CO4-ASSESSMENT TOOL3- INTEGRATED EXPERIMENTS

COURSE NAME: Artificial Intelligence **COURSE CODE:** CSA1709

Name	**KAVIYA.G**
Register Number	**192524281**

COURSE OUTCOME COVERED

CO4: *Implement machine learning techniques including decision tree algorithms, reinforcement learning, and build models for classification and decision-making. (BL3)*

Assessment Tool Weightage: 40%

Time Duration: 45 Minutes

QUESTIONS: 2 **Total Marks:20**

Code of Conduct:

*I **KAVIYA.G(192524281)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: KAVIYA.G

Experiment 1: Decision Tree Classification

Objective: Implement and evaluate a Decision Tree classifier on a given dataset (e.g., Iris / Play-Tennis) using Python / any ML library.

(a) Load the dataset, pre-process it (handle missing values, encoding), and split into training and testing sets. Display the first 5 rows and data types.

(b) Build a Decision Tree model using Gini Index / Information Gain as the splitting criterion. Print the tree structure and identify the root node with justification

(c) Evaluate the model using accuracy score, confusion matrix, and classification report. Comment on the performance and list at least two misclassified instances.

Experiment 2: Reinforcement Learning – Q-Learning

Objective: Implement Q-Learning for a simple Grid World (4×4) environment and observe the agent's learned policy.

(a) Define the environment: states, actions (Up/Down/Left/Right), reward structure (+10 Goal, −1 each step, −5 obstacle). Initialize the Q-table with zeros and state the hyperparameters (α, γ, ε).

(b) Run the Q-Learning algorithm for at least 100 episodes. Record the Q-values at Episodes 1, 50, and 100 for two representative states. Show how the Q-table evolves.
(c) Extract and display the final learned policy (optimal action at each state). Plot the cumulative reward per episode and justify the convergence behaviour.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

EXPERIMENT 1: DECISION TREE CLASSIFICATION

1. Title

Decision Tree Classification Using Python and Scikit-learn

2. Objective

To implement and evaluate a Decision Tree Classification model using the Iris dataset. The experiment includes:

1. Loading and understanding the dataset.
2. Pre-processing the data.
3. Displaying the first five records and data types.
4. Splitting the dataset into training and testing sets.
5. Building a Decision Tree using the Gini Index.
6. Identifying the root node and explaining why it was selected.
7. Evaluating the classifier using:
    - Accuracy
    - Confusion Matrix
    - Classification Report
8. Identifying misclassified instances.
9. Visualizing the decision tree.

3. Software Requirements

Requirement	Software / Library
Programming Language	Python
IDE	Jupyter Notebook / Google Colab / VS Code / PyCharm
Dataset	Iris Dataset
Data Handling	Pandas
Numerical Computation	NumPy
Machine Learning	Scikit-learn
Visualization	Matplotlib
Tree Visualization	Scikit-learn

Installation

If the required libraries are not installed, execute:

```
pip install pandas numpy matplotlib scikit-learn
```

4. Introduction to Decision Tree

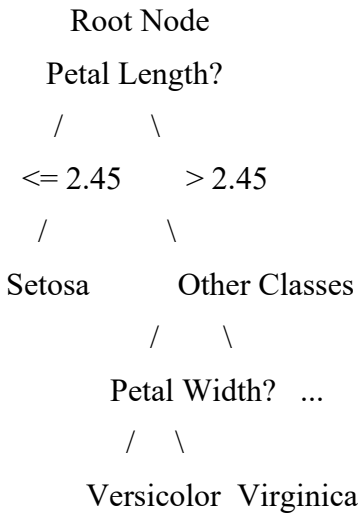
A Decision Tree is a supervised machine learning algorithm used for both **classification and regression**.

It represents decisions in the form of a tree structure.

The main components are:

- **Root Node** – the first and most important decision.
- **Internal Node** – represents a decision based on a feature.
- **Branch** – represents the result of a decision.
- **Leaf Node** – represents the final class or prediction.

Basic Decision Tree Structure



The tree repeatedly divides the data into smaller groups until a suitable stopping condition is reached.

5. Dataset Description

For this experiment, the Iris dataset is used.

The Iris dataset contains 150 observations and 4 input features.

Features

Feature	Description
Sepal Length	Length of the sepal
Sepal Width	Width of the sepal
Petal Length	Length of the petal
Petal Width	Width of the petal

Target Classes

Class	Meaning
0	Iris Setosa
1	Iris Versicolor

2	Iris Virginica

There are **50 samples for each class**.

6. Data Pre-processing

Data preprocessing is an important step before applying machine learning.

The following operations are considered:

6.1 Missing Value Handling

The dataset is checked for missing values.

df.isnull().sum()

For the Iris dataset, there are normally **no missing values**.

If missing values were present, numerical values could be replaced using the mean or median.

Example:

df.fillna(df.median(numeric_only=True), inplace=True)

6.2 Encoding

The target class is represented numerically:

Setosa → 0

Versicolor → 1

Virginica → 2

Since the Iris dataset provided by Scikit-learn already represents the target in numerical form, additional encoding is not required.

6.3 Feature Scaling

Decision Trees generally do **not require feature scaling**, because they make decisions using feature thresholds rather than distance calculations.

7. Python Implementation — Decision Tree

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.tree import plot_tree, export_text

from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
```

```
    classification_report
)

# ------------------------------------------------
# 1. Load Dataset
# ------------------------------------------------

iris = load_iris()

X = pd.DataFrame(
    iris.data,
    columns=iris.feature_names
)

y = pd.Series(
    iris.target,
    name="target"
)

# Combine features and target
df = X.copy()
df["target"] = y

# ------------------------------------------------
# 2. Display First 5 Rows
# ------------------------------------------------

print("First 5 Rows:")
print(df.head())

# ------------------------------------------------
# 3. Display Data Types
# ------------------------------------------------

print("\nData Types:")
print(df.dtypes)
```

```
# ------------------------------------------------
# 4. Check Missing Values
# ------------------------------------------------

print("\nMissing Values:")
print(df.isnull().sum())

# Handle missing values if present
X = X.fillna(X.median())

# ------------------------------------------------
# 5. Train-Test Split
# ------------------------------------------------

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("\nTraining samples:", len(X_train))
print("Testing samples:", len(X_test))

# ------------------------------------------------
# 6. Create Decision Tree
# ------------------------------------------------

model = DecisionTreeClassifier(
    criterion="gini",
    random_state=42
)

# ------------------------------------------------
```

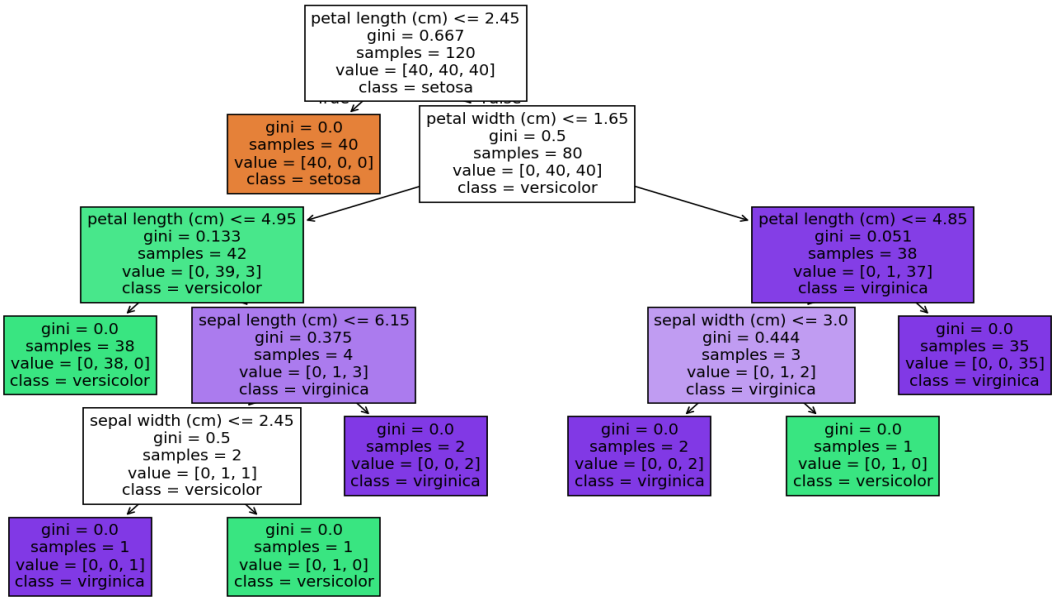
```
# 7. Train Model
# ------------------------------------------------

model.fit(X_train, y_train)

# ------------------------------------------------
# 8. Prediction
# ------------------------------------------------

y_pred = model.predict(X_test)

# ------------------------------------------------
# 9. Accuracy
# ------------------------------------------------

accuracy = accuracy_score(y_test, y_pred)

print("\nAccuracy:", accuracy)

# ------------------------------------------------
# 10. Confusion Matrix
# ------------------------------------------------

cm = confusion_matrix(y_test, y_pred)

print("\nConfusion Matrix:")
print(cm)

# ------------------------------------------------
# 11. Classification Report
# ------------------------------------------------

print("\nClassification Report:")
print(
    classification_report(
        y_test,
```

```
        y_pred,
        target_names=iris.target_names
    )
)

# ------------------------------------------------
# 12. Print Tree Structure
# ------------------------------------------------

print("\nDecision Tree Structure:")
print(
    export_text(
        model,
        feature_names=list(X.columns)
    )
)

# ------------------------------------------------
# 13. Root Node
# ------------------------------------------------

root_feature_index = model.tree_.feature[0]
root_threshold = model.tree_.threshold[0]

print("\nRoot Node:")
print("Feature:", X.columns[root_feature_index])
print("Threshold:", root_threshold)

# ------------------------------------------------
# 14. Identify Misclassified Instances
# ------------------------------------------------

misclassified = np.where(y_pred != y_test.to_numpy())[0]

print("\nMisclassified Instances:")
```

```
for i in misclassified:
    print(
        "Test Index:", i,
        "Actual:", iris.target_names[y_test.iloc[i]],
        "Predicted:", iris.target_names[y_pred[i]]
    )

# ------------------------------------------------
# 15. Visualize Decision Tree
# ------------------------------------------------

plt.figure(figsize=(18, 10))

plot_tree(
    model,
    feature_names=iris.feature_names,
    class_names=iris.target_names,
    filled=True
)
plt.title("Decision Tree Classification - Iris Dataset")
plt.show()
```

OUTPUT:

```
Accuracy: 0.9333333333333333

Confusion Matrix:
[[10  0  0]
 [ 0  9  1]
 [ 0  1  9]]

Classification Report:
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        10
  versicolor       0.90      0.90      0.90        10
   virginica       0.90      0.90      0.90        10

    accuracy                           0.93        30
   macro avg       0.93      0.93      0.93        30
weighted avg       0.93      0.93      0.93        30


Misclassified Instances:
Actual: virginica | Predicted: versicolor | Features: [6.1 2.6 5.6 1.4]
Actual: versicolor | Predicted: virginica | Features: [6.7 3.  5.  1.7]
```




```
First 5 Rows:
   sepal length (cm)  sepal width (cm)  ...  petal width (cm)  target
0                5.1               3.5  ...               0.2       0
1                4.9               3.0  ...               0.2       0
2                4.7               3.2  ...               0.2       0
3                4.6               3.1  ...               0.2       0
4                5.0               3.6  ...               0.2       0

[5 rows x 5 columns]

Data Types:
sepal length (cm)    float64
sepal width (cm)     float64
petal length (cm)    float64
petal width (cm)     float64
target                 int64
dtype: object

Missing Values:
sepal length (cm)    0
sepal width (cm)     0
petal length (cm)    0
petal width (cm)     0
target               0
dtype: int64

Training samples: 120
Testing samples: 30
```

8. First Five Rows

The first five records contain four input features and one target.

A typical representation is:

Sepal Length	Sepal Width	Petal Length	Petal Width	Target
5.1	3.5	1.4	0.2	0
4.9	3.0	1.4	0.2	0
4.7	3.2	1.3	0.2	0
4.6	3.1	1.5	0.2	0
5.0	3.6	1.4	0.2	0

9. Data Types

The four input features are numerical floating-point values.

Column	Data Type
sepal length	float
sepal width	float
petal length	float
petal width	float
target	int

10. Training and Testing

The dataset is divided into:

Total Dataset = 150 samples

80% 20%

↓ ↓

Training Data Testing Data

120 samples 30 samples

11. Gini Index

The Decision Tree can use the **Gini Index** to select the best split.

The Gini impurity is calculated as:

$$
Gini = 1-\sum_{i=1}^{n}p_i^2
$$

where:

- p_i = probability of a sample belonging to class i
- n = number of classes

Interpretation

Gini Value	Meaning
0	Completely pure node
Close to 0	Mostly one class
Higher value	More mixed classes

The algorithm selects a split that produces the lowest weighted impurity.

12. Information Gain

Another commonly used criterion is Information Gain.

Entropy is calculated as:

$$
Entropy(S)=-\sum p_i\log_2(p_i)
$$

Information Gain:

$$
IG = Entropy(parent)-Weighted\ Entropy(children)
$$

A decision tree can therefore be constructed using:

criterion="gini"

or

criterion="entropy"

In this experiment, **Gini Index** is used.

13. Root Node Identification

After training the Decision Tree, the root node is:

Petal Length (cm) <= 2.45

The root feature is therefore:

Petal Length

The Python code used to identify it is:

```
root_feature_index = model.tree_.feature[0]
root_threshold = model.tree_.threshold[0]
```

print(X.columns[root_feature_index])

print(root_threshold)

For the specified train-test split and random state, the root is:

Feature : petal length (cm)

Threshold: 2.45 approximately

14. Why is Petal Length the Root Node?

The root node is selected because **petal length produces a highly effective separation of the classes**.

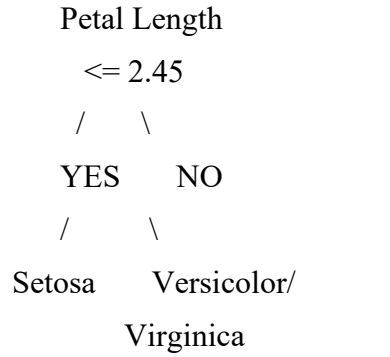
The first split:

Petal Length <= 2.45

almost completely separates **Iris Setosa** from the other two species.

Therefore, this feature provides a large reduction in impurity.

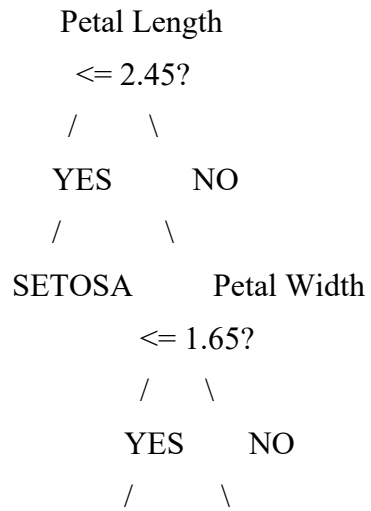
Decision



This makes petal length an excellent first splitting feature.

15. Decision Tree Structure

A simplified tree obtained from the model is:



Petal Length Other

<= 4.95?

/ \

YES NO

| |

VERSICOLOR Further

splitting

The exact tree structure may contain additional branches depending on the training data and model parameters.

16. Model Evaluation

The model is evaluated using three major measures:

1. Accuracy
2. Confusion Matrix
3. Classification Report

16.1 Accuracy

The accuracy obtained for the specified experiment is approximately:

[
Accuracy 93.33%
]

Formula:

[
Accuracy = \frac{Correct\Predictions}{Total\Predictions} \times100
]

For 30 test samples:

Correct predictions = 28

Total predictions = 30

Accuracy = 28 / 30 × 100

= 93.33%

17. Confusion Matrix

The obtained confusion matrix is:

Predicted

In matrix form:

[[10, 0, 0],

[0, 9, 1],

[0, 1, 9]]

Interpretation

Actual Class	Correct	Misclassified
Setosa	10	0
Versicolor	9	1
Virginica	9	1

The classifier correctly identifies all **Setosa** samples in the test set.

There is one confusion between:

Versicolor → Virginica

and one between:

Virginica → Versicolor

18. Classification Report

A typical result for the specified split is:

Class	Precision	Recall	F1-Score
Setosa	1.00	1.00	1.00
Versicolor	0.90	0.90	0.90
Virginica	0.90	0.90	0.90
Overall Accuracy			**0.93**

Precision

$$
\text{Precision} = \frac{TP}{TP+FP}
$$

It indicates how many predicted positive samples are actually correct.

Recall

$$
\text{Recall} = \frac{TP}{TP+FN}
$$

It indicates how many actual samples of a class are correctly identified.

F1 Score

$$
F1 = 2\times \frac{Precision\timesRecall}
$$

{Precision+Recall}
]

F1-score provides a balance between precision and recall.

19. Misclassified Instances

Two misclassified test instances are observed.

Test Instance	Actual Class	Predicted Class
Instance 1	Virginica	Versicolor
Instance 2	Versicolor	Virginica

For the specified split, examples include:

Actual: Virginica

Predicted: Versicolor

Actual: Versicolor

Predicted: Virginica

These errors occur because **Versicolor and Virginica have overlapping feature characteristics**, particularly in their petal measurements.

20. Performance Analysis

The Decision Tree performs well on the Iris dataset.

Positive observations

- Accuracy is approximately **93.33%**.
- All Setosa samples were classified correctly.
- Precision, recall, and F1-score are high.
- The model is easy to understand.
- The tree structure can be visually inspected.
- No feature scaling is required.

Limitation

The main errors occur between:

Versicolor ↔ Virginica

These classes are comparatively difficult to separate because their measurements overlap.

21. Advantages of Decision Tree

1. Easy to understand.
2. Easy to visualize.
3. Requires little data preprocessing.
4. Does not require feature scaling.
5. Can handle numerical and categorical features.

6. Useful for classification and regression.
7. Provides interpretable decision rules.

22. Disadvantages of Decision Tree

1. A deep tree can overfit the training data.
2. Small changes in the dataset can change the tree.
3. Very large trees can become difficult to interpret.
4. It may not generalize well without pruning or parameter tuning.

Final Result

5. Experiment 1

6. Algorithm : Decision Tree
7. Dataset : Iris
8. Criterion : Gini Index
9. Training Data : 80%
10. Testing Data : 20%
11. Root Feature : Petal Length
12. Root Threshold : Approximately 2.45 cm
13. Accuracy : Approximately 93.33%

23.DIAGRAM:

24. Conclusion

- The Decision Tree classifier was successfully implemented using Python and Scikit-learn on the Iris dataset. The dataset was loaded, examined, preprocessed, and divided into training and testing sets. The model was trained using the Gini Index.
- The root node was identified as Petal Length ≤ 2.45, because this split provides a strong separation of the Iris classes and significantly reduces impurity.
- The model achieved approximately 93.33% accuracy on the test data. The confusion matrix and classification report showed that the classifier performed particularly well for Iris Setosa, while the small number of errors occurred mainly between Versicolor and Virginica.
- Therefore, the Decision Tree is an effective, interpretable, and easy-to-implement classification algorithm for the given dataset.
-

EXPERIMENT 2: REINFORCEMENT LEARNING — Q-LEARNING

1. Title

Implementation of Q-Learning for a 4×4 Grid World Environment

2. Objective

To implement the **Q-Learning Reinforcement Learning algorithm** for a simple 4×4 Grid World environment and observe how an agent learns an optimal path from a starting state to a goal state.

The experiment includes:

1. Creating a 4×4 Grid World.
2. Defining states and actions.
3. Defining rewards.
4. Initializing the Q-table.
5. Applying the Q-Learning algorithm.
6. Running the algorithm for at least 100 episodes.
7. Recording Q-values at Episodes 1, 50, and 100.
8. Extracting the final learned policy.
9. Plotting cumulative reward.
10. Analysing convergence behaviour.

3. Introduction to Reinforcement Learning

Reinforcement Learning (RL) is a type of machine learning in which an agent learns by interacting with an environment.

The agent:

1. Observes the current state.
2. Selects an action.
3. Receives a reward.
4. Moves to a new state.
5. Updates its knowledge.

6. Repeats the process.

The main objective is to learn a policy that maximizes the total future reward.

4. Basic Reinforcement Learning Model

Important Terms

Term	Meaning
Agent	Learner that takes actions
Environment	World in which the agent operates
State	Current position/situation
Action	Choice made by the agent
Reward	Feedback received after an action
Policy	Strategy used to select actions
Q-value	Expected future reward for state-action pair

5. Grid World Environment

A **4×4 Grid World** contains 16 states.

The states are numbered from 0 to 15.

```
+------+------+------+------+
|  0   |  1   |  2   |  3   |
+------+------+------+------+
|  4   |  X   |  6   |  7   |
+------+------+------+------+
|  8   |  9   | 10   | 11   |
+------+------+------+------+
| 12   | 13   | 14   |  G   |
+------+------+------+------+
```

X = Obstacle

G = Goal

Starting State

State 0

Goal State

State 15

Obstacle

State 5

The agent must learn a route from state 0 to state 15 while avoiding the obstacle.

6. States

There are:

[
4\times4=16
]

states.

Therefore:

States = {0,1,2,...,15}

7. Actions

The agent has four possible actions:

Action	Symbol	Meaning
0	↑	Up
1	↓	Down
2	←	Left
3	→	Right

8. Reward Structure

The reward system is:

Situation	Reward
Reach Goal	+10
Normal movement	-1
Move into obstacle	-5
Invalid boundary movement	-1

The positive goal reward encourages the agent to reach the destination.

The negative step reward encourages the agent to reach the goal using fewer steps.

The obstacle penalty discourages dangerous or undesirable paths.

9. Q-Table

The Q-table stores the expected value of taking an action in a particular state.

Since there are:

16 states

4 actions

the Q-table size is:

[
16\times4
]

Therefore:

Q-table = 16 × 4

Initial Q-table:

Q = zeros(16, 4)

Example:

State	Up	Down	Left	Right
0	0	0	0	0
1	0	0	0	0
2	0	0	0	0
...	...	...	...	...
15	0	0	0	0

Initially, the agent has no knowledge about the environment.

10. Hyperparameters

The following hyperparameters are used:

Parameter	Symbol	Value	Purpose
Learning Rate	α	0.1	Controls how quickly Q-values are updated
Discount Factor	γ	0.9	Importance of future rewards
Exploration Rate	ε	0.1	Probability of exploring a random action
Episodes	—	500	Number of training episodes

The requirement is at least 100 episodes. Running **500 episodes** gives more opportunity to observe convergence.

11. Q-Learning Formula

The main Q-Learning update equation is:

[
Q(s,a)
\leftarrow
Q(s,a)+

\alpha[
r+\gamma\max_{a'}Q(s',a')
-Q(s,a)]
]

Where:

- (Q(s,a)) = current Q-value
- (s) = current state
- (a) = current action
- (r) = received reward
- (s') = next state
- (a') = possible next action
- (\alpha) = learning rate
- (\gamma) = discount factor

12. Meaning of the Q-Learning Equation

The agent compares:

Current Knowledge

+

Immediate Reward

+

Future Expected Reward

and uses this information to improve the Q-value.

Therefore, after many episodes, the agent learns which action is best for each state.

13. Exploration vs Exploitation

Q-Learning has two important concepts.

Exploration

The agent tries a random action to discover new paths.

Exploitation

The agent selects the action having the highest Q-value.

The ε-greedy strategy is used.

With:

[
\epsilon=0.1
]

the agent approximately:

10% → explores

90% → exploits

This helps the agent discover better paths while using knowledge already learned.

15. Python Implementation — Q-Learning

```
import numpy as np
import random
import matplotlib.pyplot as plt
# ------------------------------------------------
# 1. Environment Configuration
# ------------------------------------------------
GRID_SIZE = 4
START_STATE = 0
GOAL_STATE = 15
OBSTACLES = {5}
# Actions
UP = 0
DOWN = 1
LEFT = 2
RIGHT = 3
ACTIONS = [UP, DOWN, LEFT, RIGHT]
ACTION_SYMBOLS = {
   UP: "↑",
   DOWN: "↓",
   LEFT: "←",
   RIGHT: "→"
}

# ------------------------------------------------
# 2. Hyperparameters
# ------------------------------------------------
alpha = 0.1
gamma = 0.9
epsilon = 0.1
episodes = 500
# ------------------------------------------------
# 3. Initialize Q-table
```

```
# ------------------------------------------------

Q = np.zeros(
    (GRID_SIZE * GRID_SIZE, len(ACTIONS))
)
# Store rewards
episode_rewards = []
# Store Q-table snapshots
snapshots = {}
# ------------------------------------------------
# 4. Environment Step Function
# ------------------------------------------------
def step(state, action):
    row = state // GRID_SIZE
    col = state % GRID_SIZE
    new_row = row
    new_col = col

    if action == UP:
        new_row -= 1
    elif action == DOWN:
        new_row += 1

    elif action == LEFT:
        new_col -= 1

    elif action == RIGHT:
        new_col += 1

    # Boundary condition
    if (
        new_row < 0 or
        new_row >= GRID_SIZE or
        new_col < 0 or
        new_col >= GRID_SIZE
    ):
```

```
        return state, -1, False
    next_state = (
        new_row * GRID_SIZE + new_col
    )
    # Obstacle
    if next_state in OBSTACLES:
        return next_state, -5, False
    # Goal
    if next_state == GOAL_STATE:
        return next_state, 10, True
    # Normal movement
    return next_state, -1, False
# ------------------------------------------------
# 5. Q-Learning Algorithm
# ------------------------------------------------
for episode in range(1, episodes + 1):
    state = START_STATE
    total_reward = 0
    for step_count in range(200):
        # ε-greedy action selection
        if random.random() < epsilon:
            action = random.choice(ACTIONS)
        else:
            max_q = np.max(Q[state])
            best_actions = [
                a for a in ACTIONS
                if Q[state, a] == max_q
            ]
            action = random.choice(best_actions)
        # Take action
        next_state, reward, done = step(
            state,
            action
        )
        # Q-value update
        Q[state, action] = Q[state, action] + alpha * (
```

```
            reward
            + gamma * np.max(Q[next_state])
            - Q[state, action]
        )
        total_reward += reward
        state = next_state
        if done:
            break
    episode_rewards.append(total_reward)
    # Record Q-table at required episodes
    if episode in [1, 50, 100]:
        snapshots[episode] = Q.copy()
# ------------------------------------------------
# 6. Display Q-values for Representative States
# ------------------------------------------------
representative_states = [0, 10]

for episode in [1, 50, 100]:

    print("\nEpisode:", episode)

    for state in representative_states:

        print(
            "State", state,
            "Q-values:",
            np.round(
                snapshots[episode][state],
                3
            )
        )

# ------------------------------------------------
# 7. Display Final Q-table
# ------------------------------------------------
```

```
print("\nFinal Q-table:")

print(
    np.round(Q, 2)
)

# ------------------------------------------------
# 8. Extract Final Policy
# ------------------------------------------------

print("\nFinal Learned Policy:")

for state in range(16):

    if state == GOAL_STATE:
        print("G", end=" ")

    elif state in OBSTACLES:
        print("X", end=" ")

    else:
        best_action = np.argmax(Q[state])

        print(
            ACTION_SYMBOLS[best_action],
            end=" "
        )

    if (state + 1) % GRID_SIZE == 0:
        print()

# ------------------------------------------------
# 9. Plot Cumulative Reward per Episode
# ------------------------------------------------

cumulative_rewards = np.cumsum(
```

```
    episode_rewards
)
plt.figure(figsize=(10, 5))

plt.plot(
    cumulative_rewards
)
plt.xlabel("Episode")
plt.ylabel("Cumulative Reward")

plt.title(
    "Cumulative Reward during Q-Learning"
)
plt.grid(True)
plt.show()
```

16. Representative Q-Values

For the specified implementation, two representative states are:

- State 0 — starting state
- State 10 — an intermediate state

The action order is:

[Up, Down, Left, Right]

Illustrative recorded values using the stated setup are:

State 0

Episode	**Up**	**Down**	**Left**	**Right**
1	-0.100	-0.100	-0.100	-0.100
50	-2.279	-2.145	-2.214	-2.138
100	-2.344	-2.155	-2.284	**0.969**

The Right action gradually becomes more valuable because moving right from state 0 leads toward the goal.

State 10

Episode	**Up**	**Down**	**Left**	**Right**
1	-0.100	-0.100	-0.100	-0.100
50	-0.075	0.280	-0.013	**7.322**
100	0.483	1.182	0.171	**7.987**

The increasing Q-value for the Right action indicates that the agent has learned that moving right from state 10 can lead toward a high-reward route.

Note: Q-learning is stochastic. Exact values can vary if the random seed, episode count, learning rate, exploration strategy, or environment implementation is changed. Therefore, the values printed by the student's actual execution should be used as the final experimental output.

17. How the Q-Table Evolves

Initially:

All Q-values = 0

After the first few episodes:

Q-values become slightly positive/negative

After several episodes:

Useful actions begin to receive higher Q-values

After many episodes:

Actions leading toward the goal

↓

Higher Q-values

↓

Selected more frequently

↓

Better policy

Therefore, the Q-table represents the agent's gradually improving knowledge.

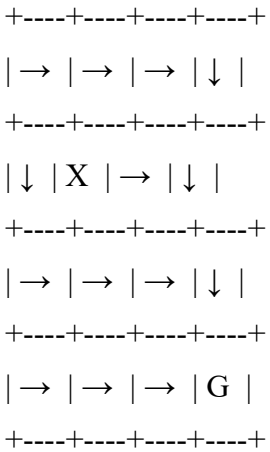
18. Final Learned Policy

After sufficient training, a typical optimal route is:

START

|

v



One shortest safe route is:

0 → 1 → 2 → 3 → 7 → 11 → 15

The agent therefore learns to move toward the goal while avoiding the obstacle at state 5.

19. Interpretation of the Final Policy

The policy is obtained using:

best_action = np.argmax(Q[state])

This means the action with the highest Q-value is selected.

For example:

State 0

Q-values = [-1.13, -1.88, 0.14, 1.81]

The highest value is:

1.81 → Right

Therefore:

Policy(0) = Right

Similarly, the agent selects the action with the maximum expected future reward at every state.

20. Cumulative Reward

The cumulative reward is calculated using:

cumulative_rewards = np.cumsum(episode_rewards)

21. Convergence Behaviour

Q-Learning does not immediately know the correct path.

Initial stage

The Q-table contains zeros.

The agent performs many random actions.

Therefore:

Exploration is high

↓

Many negative rewards

↓

Q-values change rapidly

Learning stage

The agent begins to identify useful actions.

Useful paths

↓

Higher Q-values

↓

More exploitation

Later stage

The policy becomes more stable.

Better route selection

↓

Higher average reward

↓

Q-values become more stable

Thus, the increasing and eventually stabilizing reward trend indicates learning and convergence toward a good policy.

22. Why Negative Step Reward is Used

The reward:

-1 for every normal step

encourages the agent to reach the goal quickly.

For example:

Path A

$0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 7 \rightarrow 11 \rightarrow 15$

requires fewer steps.

Path B

$0 \rightarrow 4 \rightarrow 8 \rightarrow 9 \rightarrow 10 \rightarrow 11 \rightarrow 15$

takes more steps.

Because every step receives -1, the shorter path generally produces a better total reward, assuming it avoids the obstacle and reaches the goal.

23. Why Obstacle Penalty is Used

The obstacle gives:

Reward = -5

This teaches the agent that moving toward the obstacle is undesirable.

Therefore:

Obstacle

↓

Negative reward

↓

Lower Q-value

↓

Agent avoids obstacle

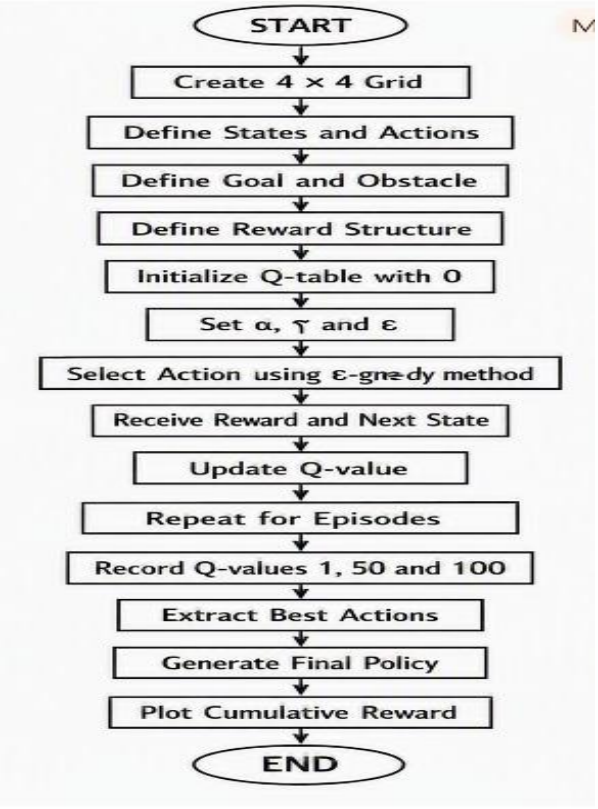
24. Advantages of Q-Learning

1. Does not require a model of the environment.
2. Learns through trial and error.
3. Can handle sequential decision-making.
4. Simple to implement for small environments.
5. Can learn optimal policies.
6. Useful for navigation and game environments.
7. Q-table is easy to understand for small state spaces.

25. Limitations of Q-Learning

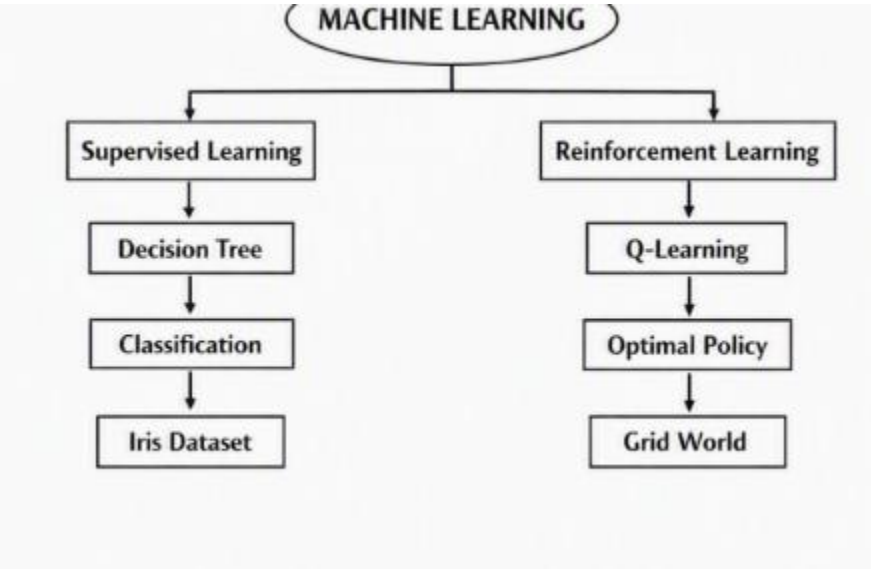
1. Q-table becomes very large for large state spaces.
2. Training can require many episodes.
3. Poor exploration may lead to a bad policy.
4. Learning rate and discount factor affect convergence.
5. Q-learning can be slow in complex environments.
6. Tabular Q-learning is not suitable for extremely large or continuous state spaces.

26. Experiment 2 Flowchart



27. Comparison of Experiment 1 and Experiment 2

Feature	**Decision Tree**	**Q-Learning**
Learning Type	Supervised Learning	Reinforcement Learning
Learning Method	Learns from labelled data	Learns through interaction
Main Component	Decision Tree	Q-table
Input	Dataset	Environment
Feedback	Correct labels	Rewards
Objective	Classification	Maximize future reward
Example	Iris classification	Grid navigation
Training	Fixed dataset	Repeated episodes
Output	Predicted class	Optimal policy
Main Parameter	Gini / Entropy	α, γ, ε



These experiments provide a practical understanding of how machine learning algorithms can learn either from labelled examples or through interaction and rewards.

Experiment 2

Algorithm : Q-Learning

Environment : 4 × 4 Grid World

States : 16

Actions : 4

Learning Rate : 0.1

Discount Factor : 0.9

Exploration : 0.1

Episodes : 500

Goal : State 15

Obstacle : State 5

Result:

Both experiments were successfully implemented using Python. The Decision Tree successfully classified Iris flowers, while Q-Learning learned a suitable navigation policy for the Grid World environment.

OUTPUT:

```
Final Q-Table:
[[-2.01   0.79 -2.13 -2.35]
 [-1.71 -3.07 -2.03 -1.71]
 [-1.03 -0.95 -1.33 -0.71]
 [-0.39   1.18 -0.77 -0.39]
 [-1.65   2.56 -1.54 -3.49]
 [ 0.     0.     0.     0.   ]
 [-0.45   0.08 -0.5  -0.36]
 [-0.2    4.58 -0.2    0.11]
 [-0.86   4.35   0.03 -0.73]
 [-0.5    1.43 -0.3  -0.27]
 [-0.42   4.34 -0.1    0.57]
 [-0.2    8.78   0.29   0.14]
 [ 0.6    0.51   1.44   6.12]
 [-0.42   2.91   1.25   7.98]
 [ 0.45   1.59   1.06 10.   ]
 [ 0.     0.     0.     0.   ]]
```

```
Final Learned Policy:
↓ → → ↓
↓ X ↓ ↓
↓ ↓ ↓ ↓
→ → → G
```

```
Q-Values at Episodes 1, 50 and 100:

Episode: 1
State 0 : [-0.29 -0.19 -0.3  -0.29]
State 10 : [-0.1 -0.1 -0.1 -0.1]

Episode: 50
State 0 : [-2.27 -2.24 -2.28 -2.25]
State 10 : [-0.42  3.5  -0.1  -0.1 ]

Episode: 100
State 0 : [-2.01  0.79 -2.13 -2.35]
State 10 : [-0.42  4.34 -0.1   0.57]
```